Facultatea de Matematică şi Informatică

Domeniul Informatică

Didactica Informaticii

Referat

Evoluția Sistemelor de Gestiune a Bazelor de Date: De la Originile Navigaționale la Modelul Matematic Relațional

Rotaru Ștefan, Grupa 234, stefan.rotaru@s.unibuc.ro

Anul Universitar 2025-2026

Cuprins

1. Introducere și Istoricul Organizării Datelor: De la Hollerith la Era Navigațională 3
2. Revoluția Matematică: Modelul Relațional (E.F. Codd, 1970) 5
2.1. Definiția Formală a unei Relații Matematice 5
2.2. Independența Datelor: Scopul Suprem al Modelului 6
2.3. Cele 12 (13) Reguli ale lui E.F. Codd (1985) 6
3. Matematica Extragerii Datelor: Algebra Relațională și Calculul Relațional 10
3.1. Algebra Relațională: Abordarea Procedurală 10
3.1.1. Operatori Fundamentali (Unari și Binari) 10
3.1.2. Operatori Derivați: Joncțiunea și Complexitatea Împărțirii () 11
3.2. Calculul Relațional și Teorema lui Codd: Logica Declarativă 13
3.2.1. Problema Expresiilor Sigure și Independența de Domeniu 14
3.2.2. Algoritmul de Reducere a lui Codd și Teorema Echivalenței (Codd's Theorem - 1972) 14
4. Teoria Normalizării: Garantarea Integrității și Structurării Datelor (1NF - 5NF) 16
5. Algoritmii de Optimizare a Interogărilor: Euristici în Algebră Relațională 20
6. Evoluția Modernă și Originalitatea Arhitecturală: Către Era AI 22
6.1. Tranziția către sistemele NoSQL (Limitările modelului Relațional) 22
6.2. SGBD în 2026: Arhitecturi Vectoriale, Evaluarea Geometrică (Cosine Similarity) și Sistemele RAG (AI) 24
6.2.1. Noua Paradigmă: Similitudinea Vectorială, Distanțe și Operatori de Bază pe Matricile Spațiului Extins (N-Dimensional) 24
7. Bibliografie 28

1. Introducere și Istoricul Organizării Datelor: De la Hollerith la Era Navigațională

Necesitatea stocării, procesării și regăsirii eficiente a datelor a precedat cu mult apariția calculatoarelor electronice moderne. Rădăcinile procesării automate a informației pot fi urmărite până la recensământul din Statele Unite din anul 1890, când Herman Hollerith a inventat primul echipament automat de procesare a informației bazat pe cartele perforate. În acea perioadă, controlul producției industriale și calculul taxelor deveniseră imposibile fără o formă de procesare automată a datelor, ceea ce a dus ulterior la fondarea companiei IBM în 1911.

Odată cu apariția primelor calculatoare comerciale (precum UNIVAC I) și tranziția de la benzi magnetice secvențiale la hard disk-uri cu acces aleator (random access) în anii 1950 și 1960, a apărut conceptul de "bază de date" pentru a elimina constrângerile hardware impuse programatorilor. În această perioadă de pionierat, datele erau gestionate prin fișiere secvențiale sau secvențial-indexate. Această abordare primară prezenta un dezavantaj critic: dependența absolută aplicație-date. Structura logică a datelor era inseparabilă de structura lor fizică pe mediul de stocare. Orice modificare a formatului fișierului sau a dispozitivului de stocare impunea rescrierea și recompilarea tuturor programelor de aplicație care accesau acele date.

Pentru a rezolva problema redundanței și a dependenței fizice, la sfârșitul anilor 1960 au fost dezvoltate primele Sisteme de Gestiune a Bazelor de Date (SGBD). Acestea au fost dominate de două modele arhitecturale majore:

1. **Modelul Ierarhic**: Acest model a fost puternic impulsionat de proiectul spațial Apollo și a fost implementat comercial cu un succes enorm de către IBM sub denumirea de IMS (Information Management System) în 1968. În cadrul acestui model, datele erau organizate sub forma unui arbore strict, în care nodurile reprezentau înregistrările de date, iar arcele reprezentau legături fizice (pointeri) de tip părinte-fiu. Deși IMS era extrem de performant pentru tranzacții bine definite, modelul ierarhic avea o limitare semantică majoră: un nod fiu putea avea un singur nod părinte, făcând extrem de dificilă și nenaturală modelarea relațiilor de tip "mulți-la-mulți" (many-to-many) prezente frecvent în realitate.
2. **Modelul Rețea**: În încercarea de a standardiza accesul la date, Departamentul Apărării din SUA a format comitetul CODASYL (Conference on Data Systems Languages). Pe baza lucrărilor lui Charles Bachman (care a dezvoltat sistemul IDS - Integrated Data Store în 1961), a fost formalizat modelul rețea. Acest model permitea înregistrărilor să fie structurate sub forma unui graf orientat generalizat, unde o înregistrare "membru" (copil) putea avea mai mulți "proprietari" (părinți),

rezolvând astfel problema asocierilor complexe din modelul ierarhic.

Dezavantajul fundamental: Navigarea. În ciuda succesului lor comercial inițial, ambele modele (ierarhic și rețea) sufereau de o deficiență arhitecturală profundă. Charles Bachman a descris faimos această paradigmă în discursul său de acceptare a premiului Turing din 1973, intitulat "The Programmer as Navigator". Pentru a accesa o anumită înregistrare, programatorul trebuia să scrie un cod procedural complex care "naviga" fizic, pas cu pas, din nod în nod, urmărind lanțurile de pointeri prin arbore sau graf. Această abordare făcea ca timpul necesar dezvoltării de noi aplicații să fie prohibitiv, iar depanarea anomaliilor de proiectare să fie o sarcină herculeană. Orice schimbare în structura legăturilor fizice ducea invariabil la prăbușirea aplicațiilor existente.

2. Revoluția Matematică: Modelul Relațional (E.F. Codd, 1970)

Ruptura definitivă de paradigma navigațională s-a produs în iunie 1970, odată cu publicarea unui articol științific de referință în revista *Communications of the ACM*, intitulat *"A Relational Model of Data for Large Shared Data Banks"*, scris de Dr. Edgar F. Codd, un matematician și cercetător la laboratorul IBM din San Jose. Codd era profund nemulțumit de faptul că utilizatorii și programatorii erau forțați să cunoască modul în care datele erau organizate fizic în mașină (reprezentarea internă).

Modelul propus de Codd a înlocuit rețelele încâlcite de pointeri fizici cu o fundamentare teoretică pură, bazată pe **teoria matematică a mulțimilor** și pe **logica cu predicate de ordinul întâi**. Această inovație a fost descrisă ulterior ca fiind una dintre cele mai mari invenții ale secolului XX în domeniul informaticii, aducându-i lui Codd premiul Turing în 1981.

2.1. Definiția Formală a unei Relații Matematice

În modelul relațional, termenul de "relație" este utilizat strict în sensul său matematic. Conform definiției originale a lui Codd, fie o colecție de mulțimi $S_1, S_2, \ldots, S_n$, care nu sunt în mod necesar distincte între ele.[3] O relație R definită pe aceste mulțimi este o submulțime a produsului lor cartezian:

$$R \subseteq S_1 \times S_2 \times \cdots \times S_n$$

Fiecare element individual al acestei submulțimi R este un n-tuplu ordonat, unde prima componentă aparține mulțimii S_1, a doua componentei S_2, și așa mai departe. Gradul relației este dat de numărul n (o relație este unară pentru gradul 1, binară pentru gradul 2, și n-ară pentru gradul n).

Pentru a transpune acest model matematic abstract în ingineria sistemelor de baze de date, se utilizează o echivalență terminologică intuitivă :

- **Relație (Relation)**: Corespunde noțiunii vizuale de *tabel* bidimensional.
- **Tuplu (Tuple)**: Corespunde unui *rând* sau unei *înregistrări* din tabel. Deoarece o relație este, prin definiție matematică, o mulțime (set), **nu pot exista tupluri duplicate** (identice pe toate dimensiunile) în cadrul aceleiași relații.
- **Atribut (Attribute)**: Corespunde unei *coloane* a tabelului.
- **Domeniu (Domain)**: Fiecare mulțime S_j este denumită domeniul de definiție al atributului respectiv, reprezentând setul tuturor valorilor atomice (indivizibile)

permise, valide și omogene pe care le poate lua acel atribut (ex. mulțimea numerelor întregi pentru vârstă, sau mulțimea șirurilor de caractere pentru un nume). Codd a introdus și distincția de "domeniu activ", reprezentând submulțimea valorilor care sunt efectiv prezente în baza de date la un moment dat.

Modelul matematic stipulează că reprezentarea sub formă de tablou (array) a unei relații trebuie să respecte proprietăți stricte: ordinea rândurilor (tuplurilor) este complet irelevantă din punct de vedere logic, toate rândurile sunt distincte, iar ordinea coloanelor este semnificativă doar în raport cu domeniile subiacente.[3]

2.2. Independența Datelor: Scopul Suprem al Modelului

Principalul obiectiv al lucrării lui Codd a fost atingerea unei stări absolute de "independență a datelor", protejând astfel investițiile uriașe ale companiilor în codul aplicațiilor împotriva schimbărilor inerente de hardware și structură internă. Modelul relațional a realizat acest deziderat eliminând sistematic trei tipuri majore de dependențe întâlnite în sistemele CODASYL și IMS:

1. **Independența față de ordonare (Ordering Dependence)**: În sistemele navigaționale, programatorii se bazau adesea pe faptul că înregistrările erau stocate fizic pe disc într-o anumită ordine (ex. sortate alfabetic). Dacă administratorul schimba această ordine fizică pentru a optimiza o altă procedură, prima aplicație eșua. Modelul relațional garantează că logica aplicației rămâne imună la ordinea fizică a tuplurilor.
2. **Independența față de indexare (Indexing Dependence)**: Indecșii (precum arborii B+) sunt structuri de date auxiliare vitale pentru accelerarea performanței la căutare. În modelul lui Codd, aplicațiile declară doar ce date doresc, fiind complet independente de existența, crearea sau distrugerea indecșilor în fundal de către administratorul sistemului.
3. **Independența față de căile de acces (Access Path Dependence)**: Eliminarea pointerilor fizici și a arborilor înseamnă că nu mai există "rute" predefinite (căi de acces) pe care un program trebuie să le traverseze obligatoriu. Orice dată este accesibilă la fel de ușor prin combinarea numelui tabelului, a cheii primare și a atributului dorit.

Această separare fundamentală a permis dezvoltarea unor limbaje de interogare de nivel înalt (precum SQL), de tip declarativ, unde utilizatorul se concentrează pe definirea rezultatului, lăsând pe seama SGBD-ului decizia asupra celui mai eficient algoritm fizic de obținere a acestuia.

2.3. Cele 12 (13) Reguli ale lui E.F. Codd (1985)

Până la mijlocul anilor 1980, succesul teoretic al modelului relațional era atât de mare încât practic fiecare vânzător de baze de date (indiferent de arhitectura sa internă reală) susținea în campaniile de marketing că vinde un "sistem relațional". Pentru a combate acest fenomen de diluare și falsificare a viziunii sale, E.F. Codd a publicat în octombrie 1985, în revista *Computerworld*, un set exhaustiv de 12 reguli (plus o regulă fundamentală, Regula 0) care defineau cu strictețe criteriile pe care un SGBD trebuie să le îndeplinească pentru a fi considerat "complet relațional" (RDBMS).

Acest manifest a devenit piatra de temelie pentru standardizarea industriei de profil :

- **Regula 0: Regula Fundației (The Foundation Rule)**: Orice sistem care pretinde a fi un sistem de gestiune a bazelor de date relaționale trebuie să fie capabil să gestioneze baza de date *exclusiv* prin facilitățile sale relaționale. Sistemul nu are voie să ceară utilizatorilor să revină la funcții non-relaționale de nivel scăzut pentru a obține o performanță acceptabilă.
- **Regula 1: Regula Informației (The Information Rule)**: Toate informațiile dintr-o bază de date relațională, fără excepție, trebuie să fie reprezentate explicit la nivelul logic într-un singur mod: ca valori atomice aflate la intersecția dintre o coloană și un rând într-un tabel. Aceasta se aplică inclusiv metadatelor (structura tabelelor), care trebuie stocate tot ca tabele.
- **Regula 2: Accesul Garantat (Guaranteed Access Rule)**: Fiecare valoare atomică din baza de date este garantat a fi accesibilă logic prin recurgerea la o combinație precisă formată din: Numele Tabelului + Valoarea Cheii Primare + Numele Coloanei. Aceasta este o formă pură de adresare asociativă, care înlocuiește scanarea fizică a adreselor de pe disc.
- **Regula 3: Tratarea Sistematică a Nulurilor (Systematic Treatment of Null Values)**: Un SGBD autentic trebuie să recunoască și să suporte nativ conceptul de valoare NULL (care reprezintă o informație "lipsă", "necunoscută" sau "inaplicabilă"), distinctă de zero, spații goale sau alte caractere, și trebuie să o manipuleze în mod consecvent și independent de tipul de date.
- **Regula 4: Catalog Dinamic Online bazat pe Modelul Relațional (Dynamic Online Catalog)**: Dicționarul de date (catalogul sistemului) trebuie să fie structurat tot sub formă de tabele relaționale, permițând utilizatorilor autorizați să interogheze metadatele folosind exact același limbaj relațional pe care îl folosesc pentru datele obișnuite (ex. SELECT * FROM information_schema.tables).
- **Regula 5: Regula Sublimbajului de Date Comprehensiv (Comprehensive Data Sublanguage Rule)**: Sistemul poate suporta mai multe limbaje, dar cel puțin unul dintre ele trebuie să fie comprehensiv și capabil să exprime: definirea datelor (DDL), definirea vizualizărilor (Views), manipularea datelor interactive și din programe (DML), constrângerile de integritate, autorizarea/securitatea și limitele

tranzacțiilor (Begin, Commit, Rollback).

- **Regula 6: Regula Actualizării Vizualizărilor (View Updating Rule)**: Orice vizualizare (view logic creat pe baza unor tabele) care este teoretic actualizabilă din punct de vedere matematic, trebuie să poată fi actualizată și de către sistemul SGBD (modificările aplicate pe view trebuie propagate corect în tabelele de bază).
- **Regula 7: Operații de nivel înalt (High-Level Insert, Update, and Delete)**: Capacitatea de a trata o relație (un set de rânduri) ca pe un singur operand nu se aplică doar la interogări, ci și la operațiunile de Inserare, Modificare și Ștergere. Acest lucru permite sistemului să optimizeze masiv căile de acces în timpul execuției.
- **Regula 8: Independența Fizică a Datelor (Physical Data Independence)**: Programele de aplicație și terminalele trebuie să rămână complet neafectate din punct de vedere logic atunci când se fac schimbări în reprezentările de stocare internă sau în metodele de acces (creare/ștergere indecși, mutare fișiere pe alte discuri).
- **Regula 9: Independența Logică a Datelor (Logical Data Independence)**: Programele de aplicație trebuie să rămână neafectate atunci când se fac schimbări (care conservă informația) la tabelele de bază. De exemplu, dacă un tabel este descompus în două tabele normalizate legate printr-o cheie străină (fără a pierde informație), un view creat peste cele două ar trebui să mențină vechile aplicații funcționale.
- **Regula 10: Independența Integrității (Integrity Independence)**: Constrângerile de integritate specifice unei baze de date (Integritatea Entității, Integritatea Referențială, constrângeri de domeniu) trebuie să poată fi definite în limbajul relațional (ex. prin clauze CHECK, PRIMARY KEY, FOREIGN KEY) și stocate direct în catalogul central al sistemului, și **nu** scrise ca reguli (IF/THEN) în programele de aplicație. Acest lucru garantează că nicio aplicație sau interfață alternativă nu poate scurtcircuita validările de securitate și coerență ale afacerii.
- **Regula 11: Independența Distribuției (Distribution Independence)**: Un SGBD trebuie să ofere unui utilizator final impresia absolută că datele se află centralizate într-un singur loc, chiar dacă în realitate, la nivel fizic, baza de date este distribuită și partiționată pe multiple servere aflate în locații geografice diferite. Rețeaua și partiționarea trebuie să fie total transparente.
- **Regula 12: Regula Non-subversiunii (The Nonsubversion Rule)**: Dacă un sistem relațional permite accesul printr-o interfață de nivel scăzut (record-at-a-time, navigare API), acea interfață ocolitoare nu trebuie sub nicio formă să poată fi utilizată pentru a submina regulile de integritate și securitate impuse prin limbajul relațional de nivel înalt.

Aceste 12 precepte au redefinit standardele de inginerie software, asigurând fundamentul pe care s-au clădit sistemele comerciale uriașe din deceniile următoare (Oracle, DB2, PostgreSQL). Deși puține sisteme îndeplinesc absolut toate regulile la modul ideal , ele reprezintă baremul de evaluare a gradului de rafinament al unui SGBD.

3. Matematica Extragerii Datelor: Algebra Relațională și Calculul Relațional

Odată ce datele sunt structurate ca mulțimi, extragerea informației necesită dezvoltarea unor limbaje matematice abstracte, capabile să opereze exclusiv pe aceste mulțimi. Modelul relațional a propus și a fundamentat matematic două astfel de sisteme formale: Algebra Relațională și Calculul Relațional. Ambele familii de limbaje sunt concise, lipsite de elementele de sintaxă "greoaie" ale limbajelor comerciale de interogare, și oferă baza teoretică pentru procesarea oricărei cereri într-un SGBD.

O proprietate fundamentală care trebuie subliniată este noțiunea de "Limbaj Relațional Complet" (Relationally Complete). Pentru ca un limbaj comercial (cum ar fi SQL - Structured Query Language) să fie considerat complet din punct de vedere relațional, el trebuie să fie capabil să exprime, ca minim absolut, orice interogare care poate fi formulată în Calculul Relațional.

3.1. Algebra Relațională: Abordarea Procedurală

Algebra Relațională, introdusă tot de E.F. Codd în 1970, este un sistem **procedural**. Aceasta înseamnă că interogarea scrisă sub forma unei expresii algebrice funcționează ca un algoritm: descrie pas cu pas **cum** trebuie să fie obținut rezultatul dorit, printr-o secvență strict ordonată de aplicare a operatorilor.

O caracteristică esențială a algebrei relaționale este **proprietatea de închidere algebrică (Closure)**. Fiecare operator algebric primește ca argumente (operanzi) una sau două relații și produce, invariabil, ca rezultat final tot o nouă relație validă. Această uniformitate matematică este extraordinar de puternică, permițând compunerea și imbricarea expresiilor la infinit (rezultatul unui operator devine operand pentru următorul, la fel ca în aritmetica de bază).

Operațiile algebrei relaționale tradiționale se pot structura algoritmic în categorii fundamentale:

3.1.1. Operatori Fundamentali (Unari și Binari)

1. **Selecția (σ)**: Este un operator unar care extrage un subset *orizontal* de tupluri (rânduri) dintr-o relație R, mai precis acele tupluri care satisfac o condiție logică predicativă C (sau P).
   - Definiție formală: $\sigma_C(R) = \{t \in R \mid t \text{ satisface } C\}$.
   - Condiția poate include operatori de comparație ($<, >, =, \neq$) și operatori logici booleeni ($\wedge, \vee, \neg$).[12]
   - Schema relației rezultate este complet identică cu schema relației de intrare. În limbajul comercial SQL, acest operator corespunde logic evaluării clauzei

WHERE.

2. **Proiecția (π)**: Reprezintă un operator unar care extrage un subset *vertical* dintr-o relație, păstrând doar atributele (coloanele) specificate în listă și eliminându-le pe celelalte.
   - Definiție formală pentru atributele $A_1 \dots A_n$: $\pi_{A_1,\dots,A_n}(R) = \{(a_1,\dots,a_n) \mid \exists t \in R \text{ unde } a_i \text{ este valoarea atributului } A_i \text{ din } t\}$.
   - O proprietate absolut esențială, decurgând din teoria pură a mulțimilor, este aceea că **proiecția trebuie să elimine automat tuplurile duplicate** din rezultatul final. Dacă din tabelul de bază extragem doar coloana "Oraș", iar în baza de date există 100 de persoane din București, rezultatul algebrei relaționale va conține cuvântul "București" o singură dată.
   - **Produsul Cartezian ($\times$)**: Este un operator binar fundamental care combină necondiționat informațiile din două relații R și S. Rezultatul este o nouă relație vastă care conține toate perechile posibile obținute prin concatenarea fiecărui tuplu din R cu absolut fiecare tuplu din S. Dacă R are N tupluri și S are M tupluri, produsul cartezian generează o relație cu M * N tupluri.
3. **Operații pe Mulțimi: Reuniunea ($\cup$), Intersecția ($\cap$) și Diferența ($-$)**: Acestea sunt preluate direct din matematică. O condiție vitală este ca relațiile operande să fie "compatibile la uniune" (union-compatible). Compatibilitatea implică două cerințe inechivoce: relațiile trebuie să aibă exact același număr de atribute (același grad), iar domeniile atributelor aflate pe poziții corespunzătoare trebuie să fie compatibile (același tip de date).
   - Reuniunea ($R \cup S$): Conține tuplurile prezente în R, în S, sau în ambele, duplicatele fiind eliminate.
   - Diferența ($R - S$): Returnează exclusiv tuplurile care se găsesc în relația R, dar care lipsesc cu desăvârșire din relația S.
4. **Redenumirea (ρ)**: Permite redenumirea (asignarea unui alias) atributelor sau a relației însăși, necesară pentru a rezolva conflictele de nume atunci când se realizează produsul cartezian al unei tabele cu ea însăși (self-join).

3.1.2. Operatori Derivați: Joncțiunea și Complexitatea Împărțirii ($\div$)

Pornind de la cei șase operatori de bază, se pot defini operatori derivați care simplifică substanțial scrierea expresiilor.

- **Joncțiunea (Join - $\bowtie$)**: Joncțiunea condițională ($\bowtie_c$) este o scurtătură matematică perfect echivalentă cu un Produs Cartezian masiv, urmat imediat de o operație de Selecție pe baza unei condiții specifice de legătură: $R \bowtie_c S = \sigma_c(R \times S)$.[14] Joncțiunea naturală (Natural Join) combină tuplurile care prezintă valori absolut identice pe toate atributele cu același nume din cele două relații, eliminând automat din rezultat una dintre coloanele redundante de legătură.
- **Împărțirea Relațională (Division - $\div$)**: Acesta este considerat, în mod unanim, cel mai provocator și complex operator algebric din punct de vedere pedagogic și practic, fiind deseori tratat superficial în cursurile introductive. Împărțirea este opusul (inversul matematic) al produsului cartezian. Dacă produsul cartezian multiplică dimensiunile, împărțirea le reduce. Operatorul identifică valorile atributelor dintr-o relație divident care sunt asociate cu absolut *toate* valorile prezente într-o relație divizor. Exemplul clasic din literatura de specialitate vizează interogări care conțin cuantificatorul universal "pentru toți": "Identifică toți furnizorii care ne livrează *toate* piesele de culoare roșie", sau "Găsește toți studenții care au absolvit *toate* cursurile din departamentul de Informatică".

  Matematic, operatorul de diviziune $R \div S$ (unde R are atributele $\{X, Y\}$ și S are atributul $\{Y\}$) se exprimă prin intermediul diferenței, proiecției și produsului cartezian, folosind o logică bazată pe o tactică a negației - căutarea valorilor care *nu* sunt răspunsuri:

$$R \div S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$$

  Explicația demonstrației:

  1. $\pi_X(R) \times S$: Se generează toate combinațiile posibile ("idealul") între toți furnizorii X și toate piesele S.[16]
  2. $(\pi_X(R) \times S) - R$: Din acest ideal perfect se scade realitatea din baza de date (R). Ce rămâne reprezintă exact asocierile "lipsă" (furnizorii care au ratat să livreze măcar o piesă din lista obligatorie).[16]
  3. $\pi_X(\dots)$: Extragem doar numele acestor furnizori deficienți.[16]
  4. $\pi_X(R) - \pi_X(\dots)$: Din totalitatea furnizorilor, scădem lista furnizorilor deficienți (cei care au lipsuri). Ce ne rămâne? Doar cei perfecți, care au livrat

tot.[16]

Deoarece limbajul SQL nu posedă un operator nativ echivalent pentru DIVIDE BY (așa cum are pentru UNION sau JOIN), formularea acestei interogări matematice într-un listing SQL real este descrisă ca o "monstruozitate". Rezolvarea prin abordare logică intuitivă presupune numărarea cardinalității mulțimilor (comparând dacă numărul de asocieri unice per grup corespunde cu totalitatea elementelor cerute):SQL

```
-- Implementarea Împărțirii Algebrice în SQL prin compararea cardinalității (Numărare)
SELECT Tabel_Dividend.Atribut_X
FROM Tabel_Dividend
WHERE Tabel_Dividend.Atribut_Y IN (SELECT Atribut_Y FROM Tabel_Divizor)
GROUP BY Tabel_Dividend.Atribut_X
HAVING COUNT(DISTINCT Tabel_Dividend.Atribut_Y) = (SELECT COUNT(*) FROM
Tabel_Divizor);
```

3.2. Calculul Relațional și Teorema lui Codd: Logica Declarativă

În contrast direct cu natura procedurală a algebrei, Calculul Relațional este un formalism pur **declarativ** (non-procedural). În limbajele declarative, utilizatorul specifică exclusiv *ce anume* dorește ca rezultat (care sunt proprietățile pe care trebuie să le îndeplinească răspunsul), fără să se preocupe absolut deloc de *cum* (ordinea operațiilor sau algoritmul fizic) va fi executată cererea de către calculator.

Calculul relațional își are rădăcinile puternic ancorate în **logica cu predicate de ordinul întâi (First-Order Logic - FOL)**. Există două ramuri (variante) ale acestui calcul:

1. **Calculul Relațional pe Tupluri (Tuple Relational Calculus - TRC)**: Aici, variabilele utilizate în formule iau valori direct peste rânduri întregi (tupluri) dintr-o relație. O interogare abstractă în TRC ia forma canonică {t | P(t)}, care se citește matematic astfel: "Găsește mulțimea tuturor tuplurilor t astfel încât predicatul (formula logică) P(t) evaluat pentru acel tuplu este Adevărat". Formulele P(t) se construiesc folosind atomi de tipul $t \in R$, operatori logici standard (conjuncție $\wedge$, disjuncție $\vee$, negație $\neg$, implicație $\Longrightarrow$) și cuantificatori universali ($\forall$) sau existențiali ($\exists$). TRC stă la baza definirii comerciale a blocurilor de selecție din standardul SQL. Deoarece operează pe rânduri complete, este considerat mult mai intuitiv din perspectivă inginerească.
2. **Calculul Relațional pe Domenii (Domain Relational Calculus - DRC)**: În mod alternativ, variabilele în DRC nu se mai leagă de un tuplu întreg, ci iau valori atomice direct din domeniile de definiție ale coloanelor individuale (atribute).

Interogările au forma abstractă $\{\langle x_1, x_2, \ldots, x_n \rangle \mid P(x_1, x_2, \ldots, x_n)\}$, evaluând condițiile strict pe componentele izolate. Abordarea DRC a format mai degrabă fundamentul teoretic pentru interfețele vizuale de tip QBE (Query-By-Example).

3.2.1. Problema Expresiilor Sigure și Independența de Domeniu

O problemă subtilă, dar fundamentală din punct de vedere logic, a apărut la evaluarea calculului. O expresie pură din calculul cu predicate de ordinul unu poate duce, teoretic, la rezultate infinite. De exemplu, formula $\{t \mid \neg R(t)\}$ reprezintă solicitarea "afișează toate tuplurile imaginabile din univers care **nu** aparțin tabelului R". Evident, un sistem informatic nu poate calcula sau stoca o infinitate de variante.

Pentru a elimina aceste anomalii, experții în teoria bazelor de date au introdus conceptul de "independență a domeniului" (Domain Independence) și "expresii sigure" (Safe Expressions). O interogare este sigură (safe) și independentă de domeniu dacă și numai dacă rezultatul evaluării ei este finit, folosind și testând exclusiv elementele și valorile care aparitează "domeniului activ" (the active domain) - adică exact setul finit de constante care sunt menționate explicit în cererea însăși sau care se găsesc efectiv stocate fizic în bazele de date asociate relațiilor din interogare. Așadar, teoria bazelor de date operează pe un subset restricționat și aplicabil practic al logicii formale.

3.2.2. Algoritmul de Reducere a lui Codd și Teorema Echivalenței (Codd's Theorem - 1972)

Având două paradigme fundamentale complet opuse ca mod de gândire – algebra (procedurală, definind algoritmii de execuție) și calculul (declarativ, orientat pe nevoile umane de exprimare) – era imperios necesar să se demonstreze legătura dintre ele.

În anul 1972, în lucrarea *“Relational Completeness of Data Base Sublanguages”*, E.F. Codd a publicat o demonstrație matematică, cunoscută astăzi sub numele de **Teorema lui Codd**. Teorema postulează o proprietate uluitoare a informaticii teoretice: **Algebra Relațională și Calculul Relațional (restricționat la expresiile sale sigure/independente de domeniu) sunt exact echivalente din punct de vedere al puterii de expresie**.

Aceasta înseamnă că orice cerere care poate fi imaginată și formulată procedural folosind arbori algebrici, poate fi întotdeauna formulată și printr-o frază declarativă de logică cu predicate, și viceversa. Dovada matematică prezentată de Codd a fost una constructivă, fiind materializată printr-un algoritm formal cunoscut ca "Algoritmul de Reducere a lui Codd" (Codd's Reduction Algorithm). Algoritmul demonstrează precis,

mecanic, cum fiecare predicat logic, cuantificator universal ($\forall$) sau existențial ($\exists$) poate fi translatat sistematic într-o secvență corespunzătoare de operatori algebrici.

Importanța practică a teoremei lui Codd pentru industria IT: Teorema echivalenței nu este un simplu exercițiu intelectual. Ea reprezintă chiar fundamentul modului de operare al SGBD-urilor moderne (Oracle, SQL Server, PostgreSQL). Pentru că cele două limbaje sunt matematic echivalente, un sistem informatic poate oferi utilizatorilor umani un limbaj de interogare prietenos, declarativ, bazat pe calculul relațional (cum este SQL - "spune-mi ce vrei"), lăsând sarcina dificilă a construirii algoritmului de regăsire unui compilator intern (Parser și Optimizer). Acest compilator intern execută de fapt o variantă a algoritmului de reducere a lui Codd, traducând automat interogarea SQL declarativă a omului într-un arbore de decizie intern format din ecuații procedurale de **Algebră Relațională** care pot fi executate eficient, instrucțiune cu instrucțiune, de către procesorul mașinii (CPU).

4. Teoria Normalizării: Garantarea Integrității și Structurării Datelor (1NF - 5NF)

Modelarea structurală și definirea inițială a asocierilor se realizează frecvent folosind Diagrame Entitate-Asociere (Entity-Relationship Model - ERD), concepute în 1976 de P.S. Chen. Însă, validarea robusteții logice a tabelelor rezultate impune recurgerea la un ansamblu de proceduri și teoreme matematice denumit **teoria normalizării**. Procesul matematic al normalizării vizează îmbunătățirea designului structural prin minimizarea nivelului de redundanță a datelor încapulate, asigurarea integrității prin protejarea legăturilor, precum și evitarea apariției stărilor de inconsistență (numite formal "anomalii de actualizare") ce pot surveni pe parcursul ciclului de viață al bazei de date.

O bază de date incorect proiectată (nenormalizată) suferă din cauza unor limitări structurale severe:

- **Anomalii la Inserare (Insert Anomaly)**: Constă în incapacitatea logică de a adăuga fapte valide (o nouă informație) într-un tabel din simplul motiv că un alt fapt, necesar ca parte a cheii, este încă inexistent sau necunoscut.
- **Anomalii la Ștergere (Delete Anomaly)**: Apariția situației în care eliminarea (ștergerea) unei linii care conține un anumit fapt singular atrage după sine și pierderea accidentală, neintenționată și ireversibilă a unor informații de importanță capitală asociate altor fapte stocate conjunctural pe aceeași linie.
- **Anomalii la Modificare/Actualizare (Update Anomaly)**: Reprezintă coșmarul oricărui administrator de sistem tranzacțional. Datorită datelor duplicate (redundante), modificarea unei proprietăți care ține de o entitate singulară impune scanarea și actualizarea mecanică a sute sau mii de tupluri. Orice omisiune a motorului tranzacțional pe parcursul acestui proces repetitiv generează instantaneu și definitiv o stare de inconsistență a datelor (baza de date stochează două adevăruri diferite și opuse despre același obiect).

Mecanismul matematic riguros care ghidează întreg algoritmul procesului de normalizare este identificarea tuturor **dependențelor funcționale**. O dependență funcțională între două submulțimi de atribute X și Y (notată matematic X->Y) este îndeplinită dacă și numai dacă cunoașterea valorilor grupului X fixează, în mod inechivoc și unic, valorile grupului Y. Spunem așadar că atributul Y "depinde funcțional" de X, determinantul X fiind de regulă o cheie primară sau candidat a relației.

Ierarhia curativă a "Formelor Normale" acționează prin aplicarea, într-o succesiune tot mai strictă, a unor reguli de constrângere, procedând, dacă este cazul, la descompunerea iterativă, matematic reversibilă și fără pierderi de informație a unui tabel masiv afectat de patologii informaționale, într-o rețea coezivă formată din multiple tabele mai simple.

Treapta de Normalizare (Forma Normală)	Condiția / Cerința Matematică Fundamentală de Validare	Tipologia Redundanței și a Anomaliilor Vizate spre Eliminare
Prima Formă Normală (1NF)	Toate atributele tabelului trebuie să conțină exclusiv valori scalare (atomice / indivizibile). Niciun atribut nu poate fi un vector sau o mulțime de valori. Orice rând trebuie să poată fi identificat unic (cheie primară).	Eliminarea grupurilor repetitive și a datelor multi-valoare. Transformă structurile complet nestructurate (ex. tipar Excel) în structuri matematice relaționale valide, conformându-se modelului teoretic.
A Doua Formă Normală (2NF)	Relația trebuie să respecte cu strictețe rigorile impuse de 1NF. Suplimentar, se interzice cu desăvârșire existența oricăror dependențe funcționale parțiale față de o cheie primară compusă (orice atribut non-cheie trebuie să depindă obligatoriu de *întreaga* cheie primară, nu doar de o subdiviziune a acesteia).	Dezintegrează design-ul în care informațiile contextuale aferente unei dimensiuni (ex. adresa depozitului) sunt repetate abuziv la fiecare facturare care folosește depozitul pe post de cheie compusă.
A Treia Formă Normală (3NF)	Relația a validat standardele 2NF. Este eliminat categoric fenomenul denumit dependență tranzitivă:	Toate atributele descriptive trebuie să definească doar cheia suverană a entității. Orice concepte care au propriile lor descrieri

	absolut niciun atribut (care nu face parte dintr-o cheie) nu are permisiunea să depindă funcțional și să fie determinat de un alt atribut non-cheie (un lanț cauzal de tip $A \to B$ și $B \to C$).	secundare sunt tăiate și plasate în tabele specializate.
Forma Boyce-Codd (BCNF)	Reprezintă de fapt o reformulare teoretică mai puternică a nivelului 3NF. BCNF impune ca pentru absolut fiecare dependență funcțională de tipul $X \to Y$ prezentă activ în schemă, elementul determinant X (cel din partea stângă a ecuației) să fie obligatoriu o supercheie validă.	Rezolvă acele instanțe limită, dar extrem de periculoase, unde defectele de modificare continuă să persiste chiar și după trecerea testului 3NF, fenomene cauzate frecvent de existența unor chei candidat multiple care au segmente comune.
A Patra Formă Normală (4NF)	Respectă BCNF și cere eradicarea completă a dependențelor multi-valoare. Dacă un atribut A determină mai multe valori independente pentru B, iar simultan determină multiple valori total independente pentru C, B și C nu pot	Previne fenomenul distructiv în care un sistem generează o sumedenie de produse carteziene false și consumatoare masive de stocare la inserarea tuturor permutațiilor posibile pentru două grupuri de date aflate în independență asimptotică.

	coexista în același tabel cu A.	
A Cincea Formă Normală (5NF)	Elimină anomaliile complexe generate de prezența unor dependențe de joncțiune (Join Dependencies). Impune ca o relație să nu mai poată fi descompusă fără pierderi în relații mai mici din a căror reconstrucție prin joncțiune naturală să nu se regenereze o informație falsă.	Gestionează patologii informaționale rutiere sau semantice avansate (situații multi-condiționale tridirecționale), de regulă întâlnite izolat, care scapă spectrului dependențelor logice binare.

Tabelul 1. Sinteza completă a etapelor procesului analitic de Normalizare a Bazelor de Date și anomaliile tratate.

În mod generalizat în literatura cursurilor universitare, precum și din punct de vedere pragmatic în industria globală, procesul de aducere și validare a unei baze de date până la standardele rigide impuse de pragul A Treia Formă Normală (3NF) este pe deplin considerat satisfăcător pentru stabilitatea și durabilitatea majorității sistemelor de procesare tranzacțională din mediul de producție (OLTP - Online Transaction Processing). Formele 4NF și 5NF se aplică pentru domenii și modelări de rețea semantice de o complexitate superioară extremă.

5. Algoritmii de Optimizare a Interogărilor: Euristici în Algebră Relațională

Performanța colosală a sistemelor RDBMS se datorează tocmai fundației lor matematice invizibile utilizatorului. Când utilizatorul final rulează un pachet de interogări (declarative prin limbajul SQL), sistemul de gestiune declanșează un modul de inteligență decizională intern cunoscut drept "Optimizatorul de Cereri" (Query Optimizer). Rolul fundamental al optimizatorului este de a prelua, analiza, a transforma comanda în algebră relațională și a concepe strategia prin care efortul de calcul pe componentele I/O și CPU este redus drastic (minimizarea funcției cost).

Etapele de transformare a unei cereri SQL sunt riguroase:

1. **Maparea cererii SQL într-o structură procedurală de Algebră Relațională**.
2. **Construcția Arborelui de Evaluare (Query Tree / Execution Plan)**: Operatorii algebrici de la nivel teoretic sunt așezați ca noduri interne într-un graf (arbore inversat), în timp ce foile terminale (nodurile frunză inferioare) ale arborelui sunt reprezentate de înseși tabelele fizice (relațiile) prezente pe disc din care urmează a se extrage blocuri de informații. Optimizatorul navighează prin fluxul de deducții matematice construind de jos în sus pașii evaluării.
3. **Aplicarea Legilor de Echivalență Algebrică (Euristica Optimizării)**: Utilizând regulile specifice ecuațiilor teoriei mulțimilor, SGBD-ul caută rearanjarea secvențială a ramurilor din structura arborelui pentru a descoperi, printre zecile de rute alternative posibile cu efect logic absolut identic, pe aceea care minimizează drastic manipularea și mutarea blocurilor de date redundante tranzitorii (memorie consumată la nivel RAM).

Conform cercetărilor expuse în lucrările și materialele academice aferente (C.J. Date, Ramakrishnan, Gehrke, Velicanu, Lungu), procedurile euristice de maximă importanță, aplicate la nivel matematic pe matricea de date, se grupează în jurul următoarelor teoreme:

- **Descompunerea și Cascadarea Selecției (Sigma-Cascade)**: Optimizatorul are dreptul logic și legal să fragmenteze o conjuncție de mai multe predicate logice aglomerate pe același operator de selecție (σ), transformându-le matematic într-o secvență fluidă și ordonată formată din filtre multiple de selecție înlănțuite succesiv. Formula canonică a teoremei demonstrează că: $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$. Avantajul decurgând de aici: Evaluatorul inteligent va inspecta dicționarul sistemului și va aplica intenționat prima oară în ordinea cronologică exclusiv acel filtru de selecție (σ) care prezintă cel mai ridicat coeficient statistic de selectivitate (se alege acea proprietate care garantează

respingerea automată a celui mai mare număr de înregistrări irelevante din prima tentativă), optimizând accesul și economisind masiv timp.

- **Împingerea Anticipată a Filtrelor prin Joncțiuni (Pushing Selection over Join)**: Aceasta este considerată, prin consens, ca fiind cea mai formidabilă teoremă menită să asigure performanța într-un ecosistem relațional complex. Joncțiunile care combină date (Joins - $\bowtie$), precum și masivele Produse Carteziene ($\times$), reprezintă cu siguranță operațiunile cele mai dureroase și consumatoare de resurse pe un server, operând logic în pași ce tind asimptotic spre complexități temporale de gen $O(M \cdot N)$ pentru două seturi mari de tabele de input.
  Dacă logica frazei impune efectuarea unei joncțiuni ample urmate tardiv de un filtru aplicabil însă doar atributelor ce provin din una din tabele (R), algoritmul optimizator detectează și inversează ordinea acțiunilor împingând (push) filtrul spre profunzimea arborelui:

$$\sigma_p(R \bowtie S) = \sigma_p(R) \bowtie S$$

  Această acțiune de filtrare anticipată a elementului R limitează uluitor volumul global care va intra efectiv în coliziunea consumatoare din cadrul algoritmului secvențial Join.
- **Distributivitatea Proiecției asupra Joncțiunii (Projection Pushing)**: Analog legilor anterioare, operatorul de decupare a coloanelor proiecție (π) este împins matematic cât mai în adâncime la nivel de ierarhie a frunzelor de către optimizator. Conform postulatului teoretic al ecuațiilor: $\pi_a(R \bowtie S) = \pi_a(\pi_b(R) \bowtie \pi_c(S))$.[35] Consecința: Sistemul renunță din primul moment, înainte ca secvențele complexe să devină active, la "bagajul mort" (coloanele care, deși parte a tabelului de referință fizic R, nu sunt necesare logicii finale impuse de clauza SELECT). Amputarea timpurie degrevează colosal lățimea de bandă pe care s-ar face transportul fluxului de rețea, iar memoria tranzitorie utilizată pe parcurs se păstrează minimală.[35]

6. Evoluția Modernă și Originalitatea Arhitecturală: Către Era AI

Discursul privind "Evoluția SGBD" ar fi neterminat fără ancorarea analizelor pur procedurale din decadele anterioare în fenomenele tehnologice care bulversează prezentul curent. Peisajul bazelor de date s-a diversificat dramatic datorită necesităților stringente de scalabilitate colosală și odată cu ascensiunea noilor modele de limbaj natural.

6.1. Tranziția către sistemele NoSQL (Limitările modelului Relațional)

Avântul erei web, al rețelelor sociale gigantice, IoT-ului și aplicațiilor masiv distribuite a pus sub presiune severă principiile de fier expuse de E.F. Codd, cu precădere formele normale constrângătoare și presupunerea garanțiilor stricte de tip ACID ale modelelor relaționale clasice. Arhitectura sistemelor SQL standard era proiectată nativ spre un mediu unitar, favorizând aproape exclusiv *scalarea verticală* (adică extinderea performanței prin achiziționarea unui singur super-server mult mai scump, dotat cu memorie și procesor cu randament din ce în ce mai greu și costisitor de asigurat pe fondul limitărilor fizice curente). Atunci când volumele de date depășesc pragurile superioare posibile gestionabile tranzacțional pe nodurile izolate unice, menținerea formelor perfecte ale regulilor relaționale prin interconectare de rețea devine lentă și paralizează procesele datorită algoritmilor constrângători de tip "two-phase commit" ceruți pentru păstrarea tranzacțională.

Această falie structurală a condus indubitabil spre deceniul **NoSQL (Not Only SQL)**, dominat inițial pe partea de volume mari neuniforme (Big Data) de structuri inovative construite ex-professo din la zero cu intenții divergente. Abordarea esențială aici constă în relaxarea teoremelor matematice relaționale tradiționale și sacrificarea parțială intenționată a garanțiilor asigurate de concurența severă din ACID, favorizând nativ în schimb posibilitatea de a partiționa invizibil volume vaste de date heterogene pe grupuri (cluster) distribuite pe structuri fizice formate din zeci, sute sau chiar mii de mașini comerciale foarte ieftine, fenomen descris tehnic drept *scalare orizontală infinită* (Scale-Out).

O paralelă extrem de grăitoare se observă la confruntarea pe plan arhitectural a unui colos clasic SGBDR recunoscut (ex: PostgreSQL) cu un lider pe noul segment specific bazelor de date flexibile fundamentate exclusiv pe stocarea de unități logice arborescente tip "documente" (ex: MongoDB):

Caracteristica	Modelul Relațional	Modelul Orientat pe

Ingineresc-Structurală	Riguros (Standardul PostgreSQL)	Documente Flexibile (NoSQL - Ex. MongoDB)
Modelarea / Entitatea Fundamentală a Stocării	Date perfect structurate și ordonate metodic sub formă de Tabele bidimensionale formate obligatoriu din Linii rigide (Tupluri) și Coloane imuabile (Atribute).[36]	Modelul folosește Date Semi-Structurate cuprinse sub formă de arborescențe "Documente" masive codate prin format BSON (JSON serializat ierarhic) grupate fluid în incinte denumite "Colecții".
Integrarea Formelor Normale / Organizarea Logică	Respectarea dogmatică a normelor stabilite matematic de teoria Formelor Normale (min. 3NF). Redundanța este asimilată unei deficiențe de fond; asocierile din lumea obiectivă sunt obligatoriu decuplate și reconstituite mecanic la solicitarea cererii prin proceduri procedurale costisitoare, de tipul JOIN.-urilor.	Forma schematică prestabilită lipsește total ("Schema-less"). Paradigma îmbrățișează și favorizează redundanța cu un scop precis: încapsulează absolut toate fațetele unei singure unități direct în structura nodulară (documente ascunse), evitând costul JOIN-urilor (citire într-un pas singular).
Criteriul Fundamental pentru Optimizarea Scalării	Limitat parțial la o puternică Scalare Verticală; abordarea orizontală menținând suportul de stabilitate garantat (ACID) reclamă un volum masiv de proceduri de rutare secundare și duce spre clasa sistemelor	Arhitectura are implementată algoritmic și natural abilitatea de partiționare a elementelor pe diferite mașini independente denumită Sharding Orizontal. Susține rularea fără obstacole tehnice limitative și preia vârfurile

	comerciale specifice, cunoscute drept NewSQL.[37]	ascuțite de read/write într-un mediu nefragmentat prin rețea de microservicii distribuite fluid.[39]

Tabelul 2: Evaluare comparativă a principalelor caracteristici inginerești definitorii care separă structural modelarea clasică (bazată matematic pe rigurozitatea tabelelor, folosind PostgreSQL drept standard) față de paradigma concurentă apărută în era modernă a sistemelor nestructurate tip document (NoSQL, axată flexibil în MongoDB).

6.2. SGBD în 2026: Arhitecturi Vectoriale, Evaluarea Geometrică (Cosine Similarity) și Sistemele RAG (AI)

Elementul absolut revoluționar care a asigurat originalitatea extremă a prezentului și prin care se fundamentează viitorul apropiat din industria bazată pe volume de date stocate este întoarcerea impresionantă spre un hibrid tehnic format prin pivotarea neașteptată a clasicului motor de gestiune SGBD Relațional pentru a fuziona impecabil alături de un segment vital aparținând direct ariei Inteligentei Artificiale Generative (GenAI) - susținând masiv Modelele Mari de Limbaj (LLMs).

Aceste unelte software (LLM-uri precum ChatGPT / Claude) sunt geniale pentru deducție analitică naturală, dar poartă un stigmat structural major: suferă intens de o formă amnezică inerentă – un moment specific ("cut-off date") de înghețare izolată a cunoștințelor statice stocate mecanic în perioada formării lor.[41] Ele nu au sub nicio formă acces spontan sau natural la procedurile interne, clienții actuali, logurile organizaționale și la datele particulare confidențiale proprii (proprietare) deținute ascuns în incinta sistemelor protejate aparținând companiilor de mare capacitate. Efectul lipsei de context vital se manifestă constant prin ceea ce experții din rândurile informaticii numesc eufemistic răspunsuri distorsionate false ("halucinații de tip AI").

6.2.1. Noua Paradigmă: Similitudinea Vectorială, Distanțe și Operatori de Bază pe Matricile Spațiului Extins (N-Dimensional)

Prin introducerea tehnologiilor sub sigla pgvector, procesul de "căutare în baza de date" (interogarea prin evaluare) a suferit cel mai important salt semantic de abordare cunoscut vreodată în mediul IT. În paradigma relațională de până acum, conceptul pur era ancorat mecanic și direct doar pe baza unei ecuații elementare perfect liniare din sistemul tradițional cu operator boolean, de genul strict SELECT... WHERE cuvânt_cercetat = "Tehnologie" (care furniza un răspuns dogmatic determinist valid cu

exactitate completă și lipsit de orice flexibilitate superioară a conceptelor nuanțate care nu potriviți la literă exactă structurii căutate).

Odată însă cu aducerea noilor dimensiuni (milioane de tupluri purtând vectori imense dimensionale prin extensie logică artificială) sub lupa optimizatorului SGBD, trecem obligatoriu sub o comandă abstractă a unei ramuri extinse ce apelează calculul geometric aplicabil spațiului N-Dimensional care calculează analitic și livrează pe de altă parte procedee numite tehnic evaluări de interogare de proximitate și cu rezultate sub o marjă aproximativă cu algoritm de căutare extrem tip KNN/ANN (Approximate Nearest Neighbor Search). Optimizatorul intern trebuie așadar programat intens să poată scana un catalog gigant și să detecteze exclusiv ce alte coloane și puncte "înrudite matematic", codate din sistemul cuvintelor, se aseamănă geometric – mai explicit fiind cu totul mult mai apropiate "în același spațiu de constelație" în care s-ar reflecta prin generare însuși vectorul convertit brusc provenit de sus direct din întrebarea cuvânt formată prin voce a unui interlocutor vizitator direct (query-embedding generat pe rute API).

Aceste formule matematice, care stau strict la baza algoritmului de execuție, lucrează complet invizibil și silențios sub acțiunea unor operatori dedicați introduși complet peste logica nativă și a primitivelor structurale procedurale deja detaliate cu 50 ani înainte de părintele Codd. În PostgreSQL cu structură extinsă (pe exemplul cu pgvector), identificăm cei 3 pivoți esențiali care conduc operațiunile algoritmice asupra metricilor vectoriali care controlează proximitatea de afinități a rețelelor neurale, cu specificațiile lor rigide:

1. **Distanța Euclidiană absolută (Euclidean Distance sau L2 Norm)** – Reprezentată formal prin sintaxa operatorului particular de execuție <-> : Este definită strict, măsurând riguros algoritmul spațiului aflat efectiv și rectiliniu creat și evaluat în formă matematic-pură, în linie dreaptă dincolo de curbele topografice de pe coordonatele spațiale ce fixează poziția finală la distanță între cele două structuri obiect de entități din care e construit vectorul (vector query raportat la vector tuplu de bază memorat). Marea importanță a utilizării analitice pe distanța Euclidiană în contextul AI se atestă practic atunci când poziția spațială totală și mărimea globală, respectiv lungimea absolută la superlativ sau magnitudinea efectivă a semnalelor vectoriale conțin efectiv ca valoare o cantitate asezonată inerentă purtătoare prin definiție chiar a unei semnificații calitative critice (de exp: pentru codare poziațională efectivă asupra punctelor esențiale din universul de date tridimensionale la un moment stabilit geometric).
2. **Produsul Scalar (Inner Product sau Dot Product)** – Invocat și evaluat matematic exclusiv cu ajutorul folosirii constante a parametrului special tip <#> :

Reprezintă pur matematic rezultanta asigurat printr-o însumare generată din multiplicarea simplă și directă (pe aceeași scară) cu efect direct a coordonatelor paralele aferente extrase din ambele secvențe analizate. Acest factor funcțional de tip operator algebric vectorial devine unul de top ales a fi activat cu o eficacitate extrem de pronunțată pe zonele procesării din motoarele comerciale unde factorul rezultat numeric brut din fracții devine el însăși obiect supus utilizării de ierarhizare a relevanței (pe o scară amplă impusă de scoruri) pentru implementările imense care compun un complex modul de "Recomandare inteligentă e-commerce sau de content video", în funcție de care rețea atențională aruncă o valoare uriașă la funcțiile cu greutate ce au prioritate pe rezultate de punctaj superior la recomandări.

3. **Similaritatea și Distanța Cosinusului (Cosine Similarity / Cosine Distance)** – Activată riguros sub influența operatorului sintactic complex <=> : Formează practic regele suveran din spectrul modelelor ce asigură procesarea optimizată în ramurile masive pur dedicate interogărilor din sfera avansată de procesare a limbajului natural (NLP sau Natural Language Processing) care compune implementarea de apogeu în tot mediul tehnologic tipic RAG. Acesta posedă ca o caracteristică primordială complet distinctă faptul că pur și simplu el ignora total și suveran influențele care vin decurgând din diferențele bruște cauzate prin forța și efectul direct apărut în urma diversității din magnitudinea absolută a dimensiunilor în ecuație pe termenii vectoriali prezenți (astfel e total decuplat irelevant cât de imens este corpus-ul, adică dacă cuvântul extras inițial de AI este tras doar și complet dintr-un enunț izolat din memorie, sau decurge prin analiză sintetică dintr-un capitol de lungime enciclopedică aflat efectiv arhivat adânc în stocarea documentelor companiei în bază). Focalizarea sa supremă unică cade și analizează doar cum, și în special cu ce fidelitate de sens exact *sunt aliniate spațial* sub incidența analitică raportată direct ca axă vizată comună la unghiul direcțional, fix cele două entități obiect comparate geometric (punctul de reper ce generează conexiunea ideatică în spațiu topografic abstract al AI).

Fundația algoritmică și de bază prin logica modelului matematic, integrat acum permanent dincolo de rândurile SQL, servește fix și unic de motor capabil în asigurarea căutării efective cu grad superior care generează conexiunea noțiunilor latente la deciziile cunoașterii reținute solid în memoria tabelului Relațional; formula canonică centrală absolută analizează corelația la un unghi pur geometric format intern pe spațiu (în simbolul matematic consacrat pe coordonate cu teta vizat ca Θ) care aliniază fix direcția axelor descrisă curat și nefiltrat din cele două repere pe matricile cu referință denumite clasic A și B, adică vectorii sub forma standard a funcției:

$$\text{Cosine Similarity} = \frac{A \cdot B}{\|A\| \cdot \|B\|}$$

Pe elementele de detaliu extrase la nivelul definițiilor aplicate din zona fracțiilor:

- Membrii de calcul amplasați superior spre zona fracției aferente Numărătorului, identificați simplu a reprezenta expresia cu forma A * B, constituie și acoperă exclusiv calculul simplu la produsul scalar preluat de regulă prin acțiunea cu formula internă pentru "Dot Product".[47]
- Expresiile care intră fix în definirea bazei din interior aflate pe poziția referinței ca Numitorul pur algebric dat sub formă explicită de la algoritm ca ||A|| * ||B|| au impusă a realiza direct acțiunea tehnică cu prioritate obligatorie aplicată pe efectuarea unei normalizări forțate (care trunchiază distorsiunile date în plus prin diferența colosală creată între normele și rezultanta aplicată prin formula matematică cu rol restrictiv specific L2 norm asupra ambelor limite superioare referitoare strict de cantitatea generală și la volume și lungimile măsurate brut și mărimilor stocate purtate la purtător).

Dacă motorul evaluează corect ecuația pe matrice și detectează clar o potrivire de context de o excelență asimilată unei afinități de aliniere semantică pe sens ce conlucrează total și cu direcție identică perfect contopită (o reflexie comună), funcția unghiului trimite invariabil deasupra spre răspuns optimizat evaluat din partea SGBD-ului la nivelul asimptotic și superior absolut la exact și scor = 1; dacă algoritmul constată clar prin calcule și proiecții geometria ce prezintă deficiențe ale orientării prin completă lipsă (sau mai exact constatând o ortogonalitate geometrică ce decuplează perfect logica), sistemul preia o distanță la care o evaluează coborât tăind evaluările pur la cota scor = 0, pe când direcțiile analitice total potrivnice (sau chiar antagonismul perfect referitor la opusul sensului și orientării complet antitetice, orientate în mod pur și distinct divergent) reies cu un rezultat pe care modelul intern impus îl va fixa și îl transmite automat mai departe decăzut spre nivel pe limită asimptotic cu un scor sever la negativ și egal asimptotic la cota minimă = -1.

7. Bibliografie

1. Codd, E. F. (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377-387.
2. Codd, E. F. (1972). *Relational Completeness of Data Base Sublanguages*. In Data Base Systems, Courant Computer Science Symposia Series 6, Englewood Cliffs, N.J.
3. Codd, E. F. (1985). *Is your DBMS really relational?* & *Does your DBMS run by the rules?*. Computerworld, Vol. 19.
4. Codd, E. F. (1990). *The Relational Model for Database Management: Version 2*. Addison-Wesley.
5. Date, C. J. (2003). *An Introduction to Database Systems* (8th Edition). Addison-Wesley.
6. Silberschatz, A., Korth, H. F., Sudarshan, S. (2019/2020). *Database System Concepts* (7th Edition). McGraw-Hill Education.
7. Elmasri, R., Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th Edition). Pearson Education.
8. Berg, K. L., Seymour, T., Goel, R. (2013). *History Of Databases*. International Journal of Management & Information Systems, 17(1).
9. Ularu, E.G., Puican, F.C., Apostu, A., Velicanu, M. (2012). *Perspectives on big data and big data analytics*. Database Systems Journal, 3(4).
10. Darwen, H. (2010). *An Introduction to Relational Database Theory*. Ventus Publishing ApS.
11. Abiteboul, S., Hull, R., Vianu, V. (1995). *Foundations of Databases*. Addison-Wesley.
12. Atre, S. (1988). *Database: Structured Techniques for Design, Performance, and Management* (2nd Ed.). John Wiley & Sons.
13. Bell, D., Grimson, J. (1992). *Distributed Database Systems*. Addison-Wesley.
14. Connolly, T., Begg, C. (1998). *Database Systems: A Practical Approach to Design, Implementation, and Management*. Addison-Wesley.
15. Fotache, M. (2001). *Proiectarea bazelor de date: Normalizare şi postnormalizare. Implementări SQL şi Oracle*. Editura Polirom.
16. Gabriela Mihai - Curs SGBD